

Parcial 1: Liquidación Semanal de Empresa de Mensajería

Universidad del Quindío

Programa de Ingeniería de Sistemas y Computación

Asignatura: Programación III

Docente: Julián E. Gutiérrez Posada

Integrantes: Sharon Dahiana Ospina - Santiago Trujillo Cubides

PARTE A: DISEÑO ARGUMENTADO

1. Elección y Justificación de Colecciones

Para la solución del sistema informático de liquidación semanal, se seleccionaron las estructuras de datos nativas de Elixir basándose en sus propiedades de inmutabilidad, rendimiento y expresividad funcional, respetando la restricción de no emplear defstruct ni estructuras personalizadas:

Colección	Uso en el Sistema	Justificación Argumentada
Listas (List)	Almacenamiento y procesamiento de repartidores, zonas y servicios.	Proporcionan un manejo secuencial lineal muy eficiente. Permiten procesamientos secuenciales mediante transformaciones con el módulo Enum y comprehensions (for) sin recurrir a recursividad explícita.
Mapas (Map)	Representación de entidades (repartidores, zonas, servicios) y agregaciones diarias.	Ofrecen acceso clave-valor rápido. Permiten estructurar los datos con llaves nombradas como átomos (:codigo, :nombre, :kilometros, etc.) haciendo transparente el significado semántico de cada dato sin necesidad de structs.
Tuplas (Tuple)	Manejo de estados de validación y retornos de	Estructura liviana de tamaño fijo ideal para representar

Colección	Uso en el Sistema	Justificación Argumentada
	función.	resultados de operaciones mediante el patrón idiomático de Elixir: <code>{:ok, valor}</code> o <code>{:error, motivo}</code> . Se integran perfectamente con la coincidencia de patrones.
Keyword Lists	Parámetros de configuración opcionales en funciones complejas como <code>ranking/2</code> .	Permiten pasar argumentos opcionales indexados por átomos con una sintaxis limpia y flexible (por ejemplo: <code>[por: & &1.neto, sentido: :desc, limite: 5]</code>), permitiendo personalizar el comportamiento de ordenamiento sin sobrecargar la firma de la función.

2. Elección y Justificación de Estructuras de Control

- **with:** Se utilizó como la estructura central para la validación encadenada de los servicios en `ValidacionMensajeria`. Permite verificar de forma secuencial y ordenada las 5 reglas de negocio (`Repartidor existente` → `Zona existente` → `Día válido 1..6` → `Kilómetros 0..45` → `Retraso -30..180 min`). Detiene la evaluación inmediatamente en el primer fallo devolviendo `{:error, motivo}` sin evaluar las condiciones subsecuentes.
- **cond:** Se empleó en `LiquidacionMensajeria.valor_servicio/1` para evaluar rangos continuos de tiempo de retraso, permitiendo asociar de manera clara las bonificaciones (+8%) o penalizaciones (-10%, -25%) según el margen de puntualidad.
- **case:** Se utilizó para la toma de decisiones basada en coincidencia de patrones exacta, como el análisis de resultados de parseo de cadenas (`Integer.parse/1`, `Float.parse/1`) y la verificación de existencia de elementos en colecciones.
- **if:** Reservado para decisiones binarias simples y directas, como determinar si un repartidor aplica cobro de alquiler de bicicleta o verificar si se alcanzó la meta diaria.
- **Enum y Comprehensions (for):** Constituyen la columna vertebral de la manipulación de datos, reemplazando totalmente cualquier necesidad de recursividad mediante operaciones funcionales puras (`Enum.map`, `Enum.filter`, `Enum.reduce`, `Enum.sort_by`).

3. Separación entre Funciones Puras e Impuras

El código sigue strictly el principio de separación de responsabilidades para garantizar la testeabilidad, mantenibilidad y determinismo del software:

- **Funciones Puras:** Ubicadas en los módulos `ValidacionMensajeria`, `CalculosMensajeria`,

LiquidacionMensajeria y ReportesMensajeria. No realizan operaciones de entrada/salida (I/O) por teclado ni pantalla, ni leen archivos o estados globales variables. Reciben datos inmutables como entrada y devuelven nuevos datos o tuplas transformadas de manera 100% determinista.

- **Funciones Impuras / De Interacción:** Concentradas en Mensajeria, ComprobanteMensajeria y los auxiliares de I/O de Util/Util2. Son las únicas encargadas de solicitar datos al usuario por consola (IO.gets/1), imprimir reportes formateados en pantalla (IO.puts/1) y medir tiempos de ejecución del sistema con :timer.tc/1.

4. Separación Modular del Proyecto

El sistema se organizó jerárquicamente en 7 archivos independientes según su responsabilidad funcional:

1. datos.exs: Módulo Datos que contiene la fuente inicial de datos (10 repartidores, 4 zonas y 94 servicios entre válidos e inválidos).
2. validacion_mensajeria.exs: Módulo ValidacionMensajeria para la validación de formato y las 5 reglas de negocio de servicios.
3. calculos_mensajeria.exs: Módulo CalculosMensajeria para acumulados por día, combinación de mapas de kilómetros y la función de ordenamiento ranking/2.
4. liquidacion_mensajeria.exs: Módulo LiquidacionMensajeria con la lógica de cálculo de valor por servicio, bonificaciones por productividad y alquiler de bicicletas.
5. reportes_mensajeria.exs: Módulo ReportesMensajeria encargado de formatear e imprimir los reportes R1 a R8.
6. comprobante_mensajeria.exs: Módulo ComprobanteMensajeria para la interacción e impresión del comprobante individual detallado.
7. mensajeria.exs: Módulo principal Mensajeria que orquesta el flujo completo de la aplicación.

REPORTE R6: PUNTUALIDAD PONDERADA POR KILÓMETROS

1. Explicación Teórica

El promedio simple calcula la media aritmética de los minutos de retraso sumando todos los retrasos y dividiendo entre la cantidad total de entregas. Esto asume erróneamente que todos los servicios tienen el mismo peso operativo. En cambio, el retraso promedio ponderado multiplica el retraso de cada servicio por la distancia recorrida en ese servicio específico y divide esa suma entre el total de kilómetros recorridos por el repartidor:

$$\text{Puntualidad Ponderada} = \frac{\text{Suma}(\text{Retraso}_i * \text{Kilometros}_i)}{\text{Suma}(\text{Kilometros}_i)}$$

De esta manera, un servicio de larga distancia (por ejemplo, 40 km) tiene una influencia proporcionalmente mayor en el indicador de puntualidad que un servicio corto (por ejemplo, 2 km). Un resultado negativo indica que en promedio las entregas se realizaron antes de la hora prevista (entregas anticipadas), lo cual representa un mejor desempeño.

2. Ejemplo Demostrativo con Datos Propios del Proyecto

Analizando los datos reales de la repartidora **Sofía Rojas (M05)** registrados en datos.exs, se observan sus 6 servicios válidos con las siguientes distancias y retrasos:

- Día 1: 26 km, retraso = -5 min
- Día 2: 27 km, retraso = 5 min
- Día 3: 28 km, retraso = 12 min
- Día 4: 29 km, retraso = 35 min
- Día 5: 30 km, retraso = 2 min
- Día 6: 31 km, retraso = 2 min

Cálculo del Promedio Simple:

```
Promedio Simple = (-5 + 5 + 12 + 35 + 2 + 2) / 6
                  = 51 / 6
                  = 8.50 minutos
```

Cálculo del Promedio Ponderado:

```
Suma Ponderada = (-5 * 26) + (5 * 27) + (12 * 28) + (35 * 29) + (2 *
30) + (2 * 31)
                = -130 + 135 + 336 + 1015 + 60 + 62
                = 1478 km·min
```

```
Suma Kilómetros = 26 + 27 + 28 + 29 + 30 + 31
                  = 171 km
```

```
Promedio Ponderado = 1478 / 171
                    ≈ 8.64 minutos
```

Conclusión: Ambos promedios son diferentes (8.50 min frente a 8.64 min) porque las distancias de los servicios varían. En este caso, el servicio del Día 4 con el mayor retraso (35 min) se realizó en una distancia superior al promedio (29 km), elevando el valor ponderado final.

3. Justificación de Negocio

En el sector de mensajería y logística urbana, tiene un sentido estratégico fundamental ponderar por distancia. Un retraso en una entrega de larga distancia afecta un compromiso de transporte mayor y consume más recursos operativos de la empresa que un retraso menor en un domicilio de pocas cuadras. La métrica ponderada refleja con mayor precisión el impacto real del tiempo de entrega sobre la masa total de carga y distancia transportada.

PARTE C: INVESTIGACIÓN Y MEDICIONES

1. Configuración de ranking/2 mediante Keyword Lists

Se implementó una función versátil en `CalculosMensajeria` que recibe una Keyword List para parametrizar el criterio de ordenamiento, el sentido (ascendente/descendente) y el límite de elementos devueltos:

```
def ranking(elementos, opciones) do
  campo = Keyword.get(opciones, :por, & &1)
```

```

sentido = Keyword.get(opciones, :sentido, :desc)
limite = Keyword.get(opciones, :limite, length(elementos))

elementos
|> Util2.ordenar(sentido, campo)
|> Enum.take(limite)
end

```

Esta abstracción permite reutilizar la misma función en múltiples reportes del sistema con total elegancia:

- Para R4 (Liquidación de mayor a menor neto): `CalculosMensajeria.ranking(liquidaciones, por: &1.neto, sentido: :desc)`
- Para R5 (Repartidor con más km en un día): `CalculosMensajeria.ranking(resumen, por: &1.kilometros, sentido: :desc, limite: 1)`

2. Combinación de Operaciones con Map.merge/3

Se cuenta con los kilómetros recorridos por la empresa aliada definidos mediante el mapa:

```
empresa_aliada = %{1 => 580.5, 2 => 430, 3 => 510, 5 => 625, 7 => 180}
```

Y el mapa local calculado a partir de los datos del proyecto:

```
kilometros_locales = %{1 => 503, 2 => 503, 3 => 503, 4 => 503, 5 => 503, 6 => 503}
```

Análisis de las preguntas de investigación:

1. ¿Qué resultado produciría Map.merge/2?

Map.merge/2 resolvería los conflictos de llaves conservando ciegamente los valores del segundo mapa (`empresa_aliada`). Produciría el mapa:

```
%{1 => 580.5, 2 => 430, 3 => 510, 4 => 503, 5 => 625, 6 => 503, 7 => 180}
```

2. ¿Por qué dicho comportamiento no resuelve el problema planteado?

Porque Map.merge/2 sobrescribe y descarta por completo los kilómetros locales en los días coincidentes (1, 2, 3 y 5), en lugar de consolidar la operación conjunta de ambas empresas. La solución correcta requiere Map.merge/3 pasando una función que sume ambos valores:

```
def combinar_kilometros(kilometros_locales, kilometros_aliados) do
  Map.merge(kilometros_locales, kilometros_aliados, fn _dia,
    local, aliado -> local + aliado end)
end
```

El resultado correcto obtenido por el programa es:

```
%{1 => 1083.5, 2 => 933, 3 => 1013, 4 => 503, 5 => 1128, 6 => 503, 7 => 180}
```

3. ¿Qué sucede con el día 7 de la empresa aliada?

El día 7 no está presente en el mapa local (que solo cubre los días 1 al 6). Al no existir un conflicto de llaves, Map.merge/3 conserva automáticamente la pareja `7 => 180` en el mapa resultante sin requerir invocación de la función combinadora.

3. Mediciones con :timer.tc/1

Para medir el rendimiento de los algoritmos sin interferencia de operaciones I/O de consola, se utilizó la función nativa de Erlang `:timer.tc/1` envolviendo la ejecución del proceso de validación masiva:

```
{tiempo_validacion, {servicios_validos, rechazados}} =
  :timer.tc(fn ->
    ValidacionMensajeria.validar_lista(Datos.servicios(),
    Datos.repartidores(), Datos.zonas())
  end)
```

Resultados de medición observados:

- Tiempo de validación de los 94 registros iniciales: **≈ 3000 microsegundos** (< 3 ms).
- Comportamiento de escalabilidad: El procesamiento se ejecuta en tiempo lineal $O(N)$ respecto al número de servicios N , garantizando un rendimiento óptimo al operar totalmente en memoria sobre listas e iteraciones funcionales del módulo Enum.

SALIDA COMPLETA DEL PROGRAMA CON SUS DATOS

A continuación se presenta la salida impresa por la consola de comandos al ejecutar el programa con la totalidad de los datos de prueba y la interacción simulada del usuario:

```
LIQUIDACIÓN SEMANAL - EMPRESA DE MENSAJERÍA
```

```
Servicios válidos iniciales: 84
```

```
Servicios rechazados iniciales: 10
```

```
Tiempo de validación: 3000 microsegundos
```

```
Ingrese un servicio adicional (repartidor; zona dia; kilometros retraso) o Enter para omitir:
```

```
M03:Z2:4 22.5 -3
```

```
Servicio adicional agregado correctamente.
```

R1. SERVICIOS RECHAZADOS

```
%{dia: 1, kilometros: 10, repartidor: "M99", retraso: 0, zona: "Z1"} ->
```

```
repartidor_desconocido
```

```
%{dia: 2, kilometros: 15, repartidor: "M98", retraso: 0, zona: "Z2"} ->
```

```
repartidor_desconocido
```

```
%{dia: 1, kilometros: 10, repartidor: "M01", retraso: 0, zona: "Z99"} -> zona_desconocida
```

```
%{dia: 2, kilometros: 10, repartidor: "M02", retraso: 0, zona: "Z98"} -> zona_desconocida
```

```
%{dia: 0, kilometros: 10, repartidor: "M03", retraso: 0, zona: "Z1"} -> dia_invalido
```

```
%{dia: 7, kilometros: 10, repartidor: "M04", retraso: 0, zona: "Z1"} -> dia_invalido
```

```
%{dia: 1, kilometros: 0, repartidor: "M05", retraso: 0, zona: "Z1"} ->
```

```
kilometros_fuera_de_rango
```

```
%{dia: 1, kilometros: 46, repartidor: "M06", retraso: 0, zona: "Z1"} ->
```

```
kilometros_fuera_de_rango
```

```
%{dia: 1, kilometros: 10, repartidor: "M07", retraso: -31, zona: "Z1"} -> retraso_invalido
```

```
%{dia: 1, kilometros: 10, repartidor: "M08", retraso: 181, zona: "Z1"} -> retraso_invalido
```

```
repartidor_desconocido: 2
```

```
zona_desconocida: 2
```

```
dia_invalido: 2
```

```
kilometros_fuera_de_rango: 2
```

```
retraso_invalido: 2
```

R2. RECORRIDO POR ZONA

Centro: 758.00 km; densidad 116.62 km/km²

Sur: 742.00 km; densidad 88.33 km/km²

Norte: 818.50 km; densidad 80.25 km/km²

Occidente: 722.00 km; densidad 60.17 km/km²

R3. RECORRIDO DIARIO DE LA EMPRESA

Día 1: 503.00 km. Sí alcanzó la meta de 500 km.

Día 2: 503.00 km. Sí alcanzó la meta de 500 km.

Día 3: 503.00 km. Sí alcanzó la meta de 500 km.

Día 4: 525.50 km. Sí alcanzó la meta de 500 km.

Día 5: 503.00 km. Sí alcanzó la meta de 500 km.

Día 6: 503.00 km. Sí alcanzó la meta de 500 km.

¿Se alcanzó todos los días?: true

¿Se alcanzó al menos un día?: true

Kilómetros combinados con la empresa aliada: %{1 => 1083.5, 2 => 933.0, 3 => 1013.0, 4 => 525.5, 5 => 1128.0, 6 => 503.0, 7 => 180.0}

Map.merge/2 reemplazaría los valores repetidos; Map.merge/3 permite sumarlos.

El día 7 se conserva porque solo aparece en el mapa de la empresa aliada.

R4. LIQUIDACIÓN SEMANAL

1. Laura Gómez (M03) | km: 520.50 | servicios: \$1249725.00 | bonificaciones: \$90000.00 | alquiler: \$60000.00 | neto: \$1279725.00

2. David López (M02) | km: 498.00 | servicios: \$1188975.00 | bonificaciones: \$90000.00 | alquiler: \$0.00 | neto: \$1278975.00

3. Carlos Ruiz (M04) | km: 498.00 | servicios: \$1188975.00 | bonificaciones: \$90000.00 | alquiler: \$0.00 | neto: \$1278975.00

4. Ana Torres (M01) | km: 498.00 | servicios: \$1188975.00 | bonificaciones: \$90000.00 | alquiler: \$60000.00 | neto: \$1218975.00

5. Mateo Castro (M06) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$0.00 | neto: \$407575.00

6. Juan Pérez (M08) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$0.00 | neto: \$407575.00

7. Camila Arias (M09) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$0.00 | neto: \$407575.00

8. Andrés León (M10) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$0.00 | neto: \$407575.00

9. Sofía Rojas (M05) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$60000.00 | neto: \$347575.00

10. Valentina Díaz (M07) | km: 171.00 | servicios: \$407575.00 | bonificaciones: \$0.00 | alquiler: \$60000.00 | neto: \$347575.00

R5. MAYOR DISTANCIA POR DÍA

Día 1: Ana Torres (M01), David López (M02), Laura Gómez (M03), Carlos Ruiz (M04) con 83.00 km

Día 2: Ana Torres (M01), David López (M02), Laura Gómez (M03), Carlos Ruiz (M04) con 83.00 km

Día 3: Ana Torres (M01), David López (M02), Laura Gómez (M03), Carlos Ruiz (M04) con 83.00 km

Día 4: Laura Gómez (M03) con 105.50 km

Día 5: Ana Torres (M01), David López (M02), Laura Gómez (M03), Carlos Ruiz (M04) con 83.00 km

Día 6: Ana Torres (M01), David López (M02), Laura Gómez (M03), Carlos Ruiz (M04) con 83.00 km

Más primeros lugares (6): Laura Gómez (M03)

R6. MEJOR PUNTUALIDAD PONDERADA

Laura Gómez (M03), retraso ponderado: 8.00 minutos.

R7. TOTALES DE LA SEMANA

Total pagado: \$7382100.00

Costo promedio pagado por kilómetro: \$2427.92

R8. REPARTIDORES CON SERVICIOS EN TODAS LAS ZONAS

Ana Torres (M01)

David López (M02)

Laura Gómez (M03)

Carlos Ruiz (M04)

Sofía Rojas (M05)

Mateo Castro (M06)

Valentina Díaz (M07)

Juan Pérez (M08)

Camila Arias (M09)

Andrés León (M10)

Ingrese el código del repartidor para generar su comprobante:

M01

COMPROBANTE DE PAGO

Ana Torres - M01

Día 1: 83.00 km | servicios: \$205200.00 | bonificación: \$15000.00

Día 2: 83.00 km | servicios: \$181250.00 | bonificación: \$15000.00

Día 3: 83.00 km | servicios: \$197250.00 | bonificación: \$15000.00

Día 4: 83.00 km | servicios: \$181875.00 | bonificación: \$15000.00

Día 5: 83.00 km | servicios: \$215900.00 | bonificación: \$15000.00

Día 6: 83.00 km | servicios: \$207500.00 | bonificación: \$15000.00

Suma de servicios: \$1188975.00

Suma de bonificaciones: \$90000.00

Alquiler de bicicleta: -\$60000.00

Neto a pagar: \$1218975.00

PARTE D: BITÁCORA, CASO DE ERROR Y REFLEXIÓN

1. Bitácora de Uso de Inteligencia Artificial

Fecha / Integrante	Consulta / Asistencia Solicitada	Sugerencia de la IA	Verificación y Adaptación Realizada
24/Sep/2026 Santiago Trujillo	Estructura general para encadenar las 5 reglas de validación en Elixir devolviendo el primer motivo de rechazo.	Propuso usar funciones recursivas con coincidencia de patrones y guardas.	Adaptado: Se descartó la recursividad por restricción del parcial y se reestructuró la solución utilizando with en ValidacionMensajeria .validar_servicio/3.
25/Sep/2026 Sharon Dahiana Ospina	Diseño de la función de ordenamiento flexible para liquidaciones y reportes.	Sugirió definir un defstruct y usar librerías externas de ordenamiento.	Adaptado: Se rechazaron los structs y librerías externas. Se implementó CalculosMensajeria.ranking/2 con Keyword Lists y mapas de estructura libre sobre Enum.sort_by.
26/Sep/2026 Grupo de trabajo	Ayuda con la fórmula de combinación de mapas en Elixir para consolidar los kilómetros de dos empresas.	Propuso utilizar Map.merge/2 directamente.	Corregido: Se detectó en pruebas que Map.merge/2 borraba los datos locales. Se cambió a Map.merge/3 con una función anónima acumuladora.

2. Caso de Error Específico y Solución

Descripción del Error Encontrado:

Durante la lectura del servicio adicional ingresado por la consola (interacción con el usuario), cuando el usuario ingresaba un formato con espacios o caracteres no numéricos en los kilómetros o retraso, o cuando se presionaba la tecla Enter en blanco, el sistema presentaba excepciones de conversión o fallos en el patrón de coincidencia.

Causa Raíz:

Uso inicial de conversiones directas que asumían siempre entradas perfectas sin evaluar previamente las tuplas de error de los parsers de Elixir.

Solución Implementada:

Se implementaron funciones auxiliares de parseo seguro con `Integer.parse/1` y `Float.parse/1` apoyadas en coincidencia de patrones con `case`, garantizando que ante cualquier entrada inválida se retorne limpiamente `{:error, :formato_invalido}` o `:omitido` sin romper el flujo de ejecución ni usar `try/rescue`:

```
defp convertir_numero(texto) do
  texto = String.trim(texto)

  case Integer.parse(texto) do
    {numero, ""} -> {:ok, numero}
    _ ->
      case Float.parse(texto) do
        {numero, ""} -> {:ok, numero}
        _ -> {:error, :formato_invalido}
      end
  end
end
```

3. Reflexión Honesta sobre el Uso de IA y Aprendizaje

El desarrollo de este proyecto fue una experiencia altamente enriquecedora para profundizar en el paradigma de programación funcional con Elixir. La restricción explícita de no utilizar recursividad, `defstruct`, proyectos mix ni librerías externas nos obligó a comprender a fondo el poder declarativo de las colecciones y transformaciones con el módulo `Enum` y `comprehensions`.

Las herramientas de Inteligencia Artificial resultaron sumamente útiles como asistentes para explorar alternativas sintácticas y acelerar la estructuración de la documentación. Sin embargo, pudimos comprobar que la IA propone con frecuencia patrones prohibidos en el curso (como recursividad o `structs`) aunque se le indique lo contrario en el prompt. Nuestra labor activa como desarrolladores consistió en auditar cada línea de código, corregir los errores conceptuales de la IA y adaptar las soluciones a los requerimientos estrictos del docente Julián E. Gutiérrez Posada. Garantizamos que cada integrante del grupo comprende la totalidad del código y está capacitado para explicar y modificar cualquier función durante la sustentación en vivo.